
Amazon DynamoDB – Complete Overview

What is Amazon DynamoDB?

♦ Amazon DynamoDB is a **fully managed**, **serverless**, and **NoSQL** database service provided by AWS.

-  **High performance** at any scale (single-digit millisecond latency)
-  **Serverless** – no need to provision or manage infrastructure
-  **Scales automatically** to handle millions of requests per second
-  **Key-Value & Document** data models

Key Features of DynamoDB

Feature	Description
 Fully managed	No servers to manage – AWS handles everything
 Performance	Single-digit millisecond latency at any scale
 Security	Integrated with IAM, KMS, VPC endpoints
 Auto-scaling	Adjusts read/write capacity based on traffic

 On-Demand or Provisioned modes	Flexible billing and scaling options
 Streams	Real-time change data capture using DynamoDB Streams
 Backup & Restore	On-demand backup and Point-in-Time Recovery
 Global Tables	Multi-region, active-active replication for high availability

Data Model

DynamoDB stores data in **tables**, but without rigid schemas like SQL. It supports:

1. Primary Key (required):

- **Partition Key:** A unique identifier for each item
- or
- **Composite Key:** Partition Key + Sort Key

```
{
  "UserID": "1234",      ← Partition Key
  "OrderDate": "2024-05-01", ← Sort Key
  "Item": "Laptop",
  "Quantity": 1
}
```

2. Secondary Indexes:

- **GSI (Global Secondary Index)** – Query on non-key attributes

- **LSI (Local Secondary Index)** – Use same partition key but different sort key
-

Access Patterns

DynamoDB is **query optimized**, not scan optimized.

You typically:

- Use **GetItem** to retrieve a specific item by key
 - Use **Query** for filtered access (PartitionKey required)
 - Avoid **Scan** unless absolutely necessary (expensive)
-

Capacity Modes

1. Provisioned Mode:

- You set **Read and Write Capacity Units (RCU/WCU)**
- Good for predictable workloads
- Can enable **Auto Scaling**

2. On-Demand Mode:

- Pay per request
- Ideal for unpredictable or spiky traffic

- No need to manage capacity
-

Example Use Cases

Use Case	Why DynamoDB?
 Web session store	Low-latency, high throughput
 E-commerce shopping cart	Scalable & fast
 Mobile game leaderboard	Real-time, consistent access
 IoT sensor data storage	Flexible and schema-less
 CMS with JSON documents	Great for document-based apps

Security & Monitoring

-  **IAM** policies for fine-grained access control
 -  **VPC endpoints** for private access
 -  **Encryption at rest** using AWS KMS
 -  **CloudWatch Metrics** to monitor usage
 -  **CloudTrail** for audit logs
-

Integration with Other AWS Services

Service	Integration
 Lambda	Event-driven triggers from DynamoDB Streams
 S3	Data export or archive from DynamoDB
 API Gateway	Serverless API with DynamoDB backend
 Kinesis	Data analytics on DynamoDB streams
 Glue	ETL data from DynamoDB to data lakes

Pricing Overview

You pay for:

- **Read/Write capacity** (RCU/WCU) or per request in On-Demand
- **Data storage**
- **Streams**
- **Backups and restore**
- **Global tables** (replication cost)

 Tip: Use the **AWS Pricing Calculator** to estimate cost.

Pros & Cons Summary

Pros 

Fully managed

Cons 

Limited complex query support

Extremely scalable	Costly for frequent scans
Serverless architecture	No JOINS (like in SQL)
Fast performance	Harder to model complex relational data

Final Thoughts

DynamoDB is **ideal for applications that need scale, speed, and simplicity**. It works especially well in serverless environments, real-time apps, and when you want predictable performance at scale.

!